

LLDD BE - Job 7 SyncCompetitorToDocument

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	12 ชั่วโมง = implementation 9 + unit test 3 (30%)
Owner	Aphiwit <Bank> Khammoon
Target repository	SBP/srm-sps-spsap-sop-sgi-batch (NestJS 11 + TypeORM · schema sps_store · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน src/main.ts แล้วรับ argument ผ่าน JOB_NAME/INPUT (local) หรือ argv[3]/argv[2] (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ @Cron · ดู srm sps spsap sop sgi batch
Objective	บันทึกข้อมูลคู่แข่งเข้าเอกสาร: อ่านข้อมูลคู่แข่งล่าสุดจาก sgi_fgi_impact_competitors แล้วบันทึกเข้า sgi_document_competitors ผ่าน Document Service โดยตรง แทนการเขียนไฟล์ BPM06003O และ SFTP ไป BPM

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope {success, data} · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (x-api-key / x-user-*) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-3-ImportImpactCompetitor.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-8-CreateCompensationDocument.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

- Main class/script: document.service.syncCompetitors / (internal scheduler / service)
- Phase: C
- Output: sgi_document_competitors (DB)
- Estimate: 9 ชั่วโมง
- Argument รับผ่าน INPUT (JSON) — local ใช้ env JOB_NAME/INPUT · AWS Batch ใช้ argv[3]/argv[2] · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอบริการ (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาที่ AWS Batch scheduled event (repo ไม่มี @Cron) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึก log integration_log โดย main.ts อัปเดตโมดูล + structured log BATCH_START/BATCH_END พร้อม runId

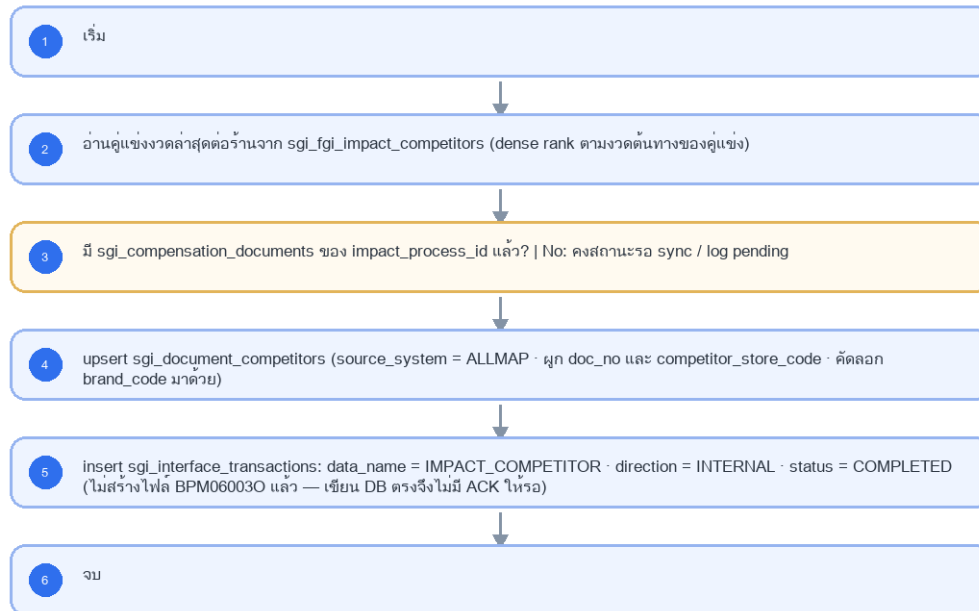
3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

4.1 Implementation Flow (ลำดับขั้นตอนการทำงาน)

LLDD BE - Job 7 SyncCompetitorToDocument

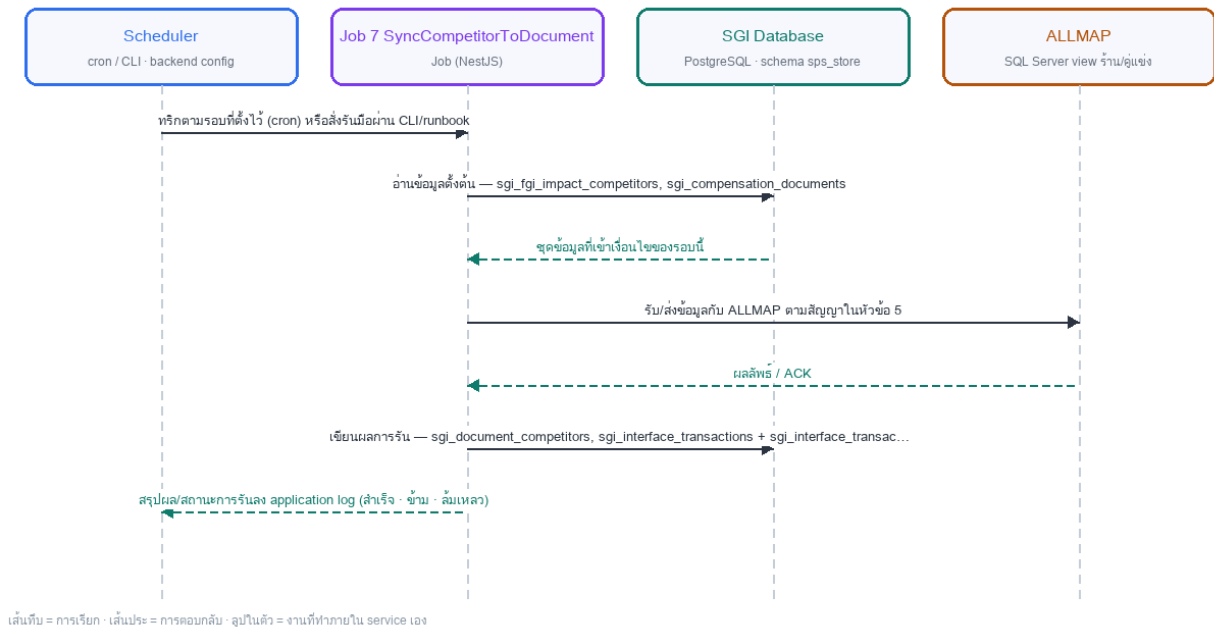


รูปที่ 1: Implementation flow reference: LLDD BE - Job 7 SyncCompetitorToDocument

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ

Sequence — LLDD BE - Job 7 SyncCompetitorToDocument



รูปที่ 2: Sequence diagram: LLDD BE - Job 7 SyncCompetitorToDocument

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 18 7-31 * *	แก้ไขได้	เหลื่อมหลัง Job 8 (17:30) เพราะต้องรอ doc_no · เวลาเหลื่อมไม่ใช่การรับประกัน — ต้องตั้ง dependency ที่ AWS Batch
Target table	sgi_document_competitors	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	upsert ด้วย doc_no / competitor_store_code · source_system = ALLMAP (ค่า canonical ค่าเดียวทั้งระบบ)
เงื่อนไขเลือกข้อมูล	งวดคู่แข่งล่าสุดต่อร้าน + forecast เริ่มต้น + ยังไม่ sync	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	คง business rule เดิม

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	FGI_IMPACT_COMPETITOR rows linked to active impact-process records and BPM/export confirmation state.
Progress	query latest competitor rows, skip already-confirmed transactions, create outbound payload per competitor, upload/export, insert confirm-receive rows.
Output	Competitor sync payload/output for downstream workflow; confirm-receive rows prevent duplicate export.

5.90 Job 7 Execution Stages

query latest competitor rows, skip already-confirmed transactions, create outbound payload per competitor, upload/export, insert confirm-receive rows.

Order	Service step	Repository	Output / failure contract
1	loadLatestDocumentCompetitors	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

Order	Service step	Repository	Output / failure contract
2	upsertDocumentCompetitors	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
3	recordInternalCompetitorSync	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
4	reconcileDocumentCompetitors	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

5.91 Job 7 Run Evidence

Evidence	Job-specific value	Acceptance
Input identity	FGI_IMPACT_COMPETITOR rows linked to active impact-process records and BPM/export confirmation state.	snapshot input file/business key/period in run record
Output identity	Competitor sync payload/output for downstream workflow; confirm-receive rows prevent duplicate export.	reconcile input, success, reject and skipped counts
Dedup proof	UNIQUE(doc_no,competitor_store_code) — คีย์เป็นสาขา ไม่ใช่เบอร์เน็ต (แก่ 2026-09-09); upsert และ prune เฉพาะ source_system=ALLMAP ให้ target ตรง source ปัจจุบันโดยไม่ลบแถว USER	rerun fixture produces no duplicate target business key
Transaction proof	upsert + prune sgi_document_competitors และ tracking (direction=INTERNAL) ใน transaction เดียวต่อ doc_no	injected failure leaves no partial committed state outside documented boundary
Security proof	service account ภายในมีสิทธิ์ SELECT source และ INSERT/UPDATE target เท่านั้น; ไม่มี external credential	config/log/error contains no plaintext secret

5.92 Legacy Java Source Reference

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ExportCompetitor.java	9-20	Legacy main entrypoint for competitor export.
fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java	659-760	Query competitor data, generate file content, upload, backup, notification.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	1596-1628	Query latest competitor rows eligible for export.

Line ranges refer to the legacy Java implementation under batchjob/fcsJar/ (path นับจากราก sbp-prototype/). Use these ranges to preserve business behavior while implementing the target Node job.

5.93 Target Repository and SQL Contract

Contract	Target implementation
Repository	documentCompetitorRepository
Idempotency / dedup	UNIQUE(doc_no,competitor_store_code) — คีย์เป็นสาขา ไม่ใช่เบอร์เน็ต (แก่ 2026-09-09); upsert และ prune เฉพาะ source_system=ALLMAP ให้ target ตรง source ปัจจุบันโดยไม่ลบแถว USER
Transaction boundary	upsert + prune sgi_document_competitors และ tracking (direction=INTERNAL) ใน transaction เดียวต่อ doc_no
Security	service account ภายในมีสิทธิ์ SELECT source และ INSERT/UPDATE target เท่านั้น; ไม่มี external credential

Input / candidate query

```
-- bind ตามลำดับ: $1=period_key
-- คัดลอกทั้ง "รหัสสาขา" และ "รหัสแบรนด์" — คัดลอกอย่างใดอย่างหนึ่งข้อมูลจะหาย
SELECT d.doc_no, c.competitor_store_code, c.brand_code, c.name_th, c.name_en, c.branch_th,
       c.zone_code, c.subzone_code, c.opened_date, c.closed_date
FROM sgi_fgi_impact_competitors c
JOIN sgi_compensation_documents d ON d.impact_process_id = c.impact_process_id
WHERE c.period_key = $1 /* period_key */;
```

Write / upsert query

```
-- bind ตามลำดับ: $1=doc_no · $2=competitor_store_code · $3=brand_code · $4=name_th · $5=name_en · $6=branch_th · $7=zone_code ·
$8=subzone_code · $9=opened_date · $10=closed_date · $11=period_key
INSERT INTO sgi_document_competitors
  (doc_no, competitor_store_code, brand_code, name_th, name_en, branch_th,
   zone_code, subzone_code, opened_date, closed_date, source_system, updated_at)
VALUES ($1 /* doc_no */, $2 /* competitor_store_code */, $3 /* brand_code */, $4 /* name_th */, $5 /* name_en */, $6 /* branch_th */,
       $7 /* zone_code */, $8 /* subzone_code */, $9 /* opened_date */, $10 /* closed_date */, 'ALLMAP', CURRENT_TIMESTAMP)
ON CONFLICT (doc_no, competitor_store_code)
DO UPDATE SET brand_code = EXCLUDED.brand_code, name_th = EXCLUDED.name_th,
              name_en = EXCLUDED.name_en, branch_th = EXCLUDED.branch_th,
              zone_code = EXCLUDED.zone_code, subzone_code = EXCLUDED.subzone_code,
              opened_date = EXCLUDED.opened_date, closed_date = EXCLUDED.closed_date,
              updated_at = CURRENT_TIMESTAMP;

DELETE FROM sgi_document_competitors dc
WHERE dc.doc_no = $1 /* doc_no */
  AND dc.source_system = 'ALLMAP'
  AND NOT EXISTS (
    SELECT 1
    FROM sgi_fgi_impact_competitors src
    JOIN sgi_compensation_documents d ON d.impact_process_id = src.impact_process_id
    WHERE d.doc_no = dc.doc_no
      AND src.period_key = $11 /* period_key */
      AND src.competitor_store_code IS NOT DISTINCT FROM dc.competitor_store_code
  );
```

5.94 Target Node Implementation

โครงสร้างนี้ระบุ service/repository เฉพาะงานและต้อง implement ตาม SQL, transaction, idempotency และ security contract ด้านบน โดยทุกชั้นต้องคืน metrics สำหรับ reconcile และ run history

```
export async function runLddBeJob7Synccompetitortodocument(ctx, services) {
  const run = await services.jobRuns.acquire({
    jobNo: "7", period: ctx.period, triggeredBy: ctx.triggeredBy
  });

  try {
    ctx = { ...ctx, runId: run.id, repository: services.documentCompetitorRepository };
    const step1 = await services.loadLatestDocumentCompetitors(ctx, undefined);
    const step2 = await services.upsertDocumentCompetitors(ctx, step1);
    const step3 = await services.recordInternalCompetitorSync(ctx, step2);
    const step4 = await services.reconcileDocumentCompetitors(ctx, step3);
    const result = step4;
    await services.jobRuns.finish(run.id, "SUCCESS", result.metrics);
    return { runId: run.id, status: "SUCCESS", ...result };
  } catch (error) {
    await services.jobRuns.finish(run.id, "FAILED", {
      errorCode: error.code ?? "JOB_FAILED",
      errorMessage: error.message
    });
    throw error;
  }
}
```

5.95 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ sgi-sync-competitor-to-document ใน src/main.ts ของ SBP/srm-sps-spsap-sop-sgi-batch โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — main.ts อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัปเดตมันจากการมี argv[3] หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env JOB_NAME	env INPUT (JSON string)	JOB_NAME=sgi-sync-competitor-to-document INPUT='{ "docNo": "2026/00123" }' npm run start

ช่องทางรับ	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
AWS Batch (ตารางเวลาจริง)	<code>process.argv[3]</code>	<code>process.argv[2]</code> (JSON string)	<code>node dist/main.js '{"docNo":"2026/00123"}' sgi-sync-competitor-to-d ocument</code>

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron/@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: ไม่รับ args — sync ทุกเอกสารที่เข้าเงื่อนไข
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
docNo	string	null = ทุกเอกสารที่เข้าเงื่อนไข	YYYY/xxxxx · ไม่พบเอกสาร = INVALID_JOB_INPUT	ของใหม่
impactMonth	string	null	YYYY-MM	ของใหม่
dryRun	boolean	false	true = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
limit	number	null	จำกัดจำนวนรายการที่ประ มวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกากการ validate ที่ทุก job ต้องทำเหมือนกัน — parse INPUT ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log BATCH_END ด้วย
batchStatus: 'FAILED' แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เจ็บ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) ·
ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ลม

- ระบุ docNo แล้ว prune ต้องจำกัดขอบเขตเฉพาะเอกสารนั้น — ห้ามลบแถวของเอกสารอื่น

5.96 เงื่อนไขตัดสินใจ (Decision Rules) — ตัดสินจากอะไร

Job 7 คัดลอกคู่แข่งจากโซน A เข้าเอกสาร — เรื่องที่ต้องชัดคือ แถวไหนลบได้ แถวไหนห้ามแตะ
เพราะเอกสารเดียวกันมีทั้งแถวที่ระบบใส่และแถวที่คนคีย์เอง

คำถามที่โค้ดต้องตอบ	ตัดสินใจ (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
เอกสารนี้ต้อง sync หรือไม่	<code>sgi_compensation_documents (doc_no · สถานะเอกสาร) + sgi_fgi_impact_competitors</code> ของรอบเดียวกัน	เอกสารที่ยังไม่จบและมีข้อมูลคู่แข่งของรอบนั้นอยู่ · ส่ง docNo มากี่จำกัดเฉพาะเอกสารนั้น (ดู 5.95)	ไม่มีข้อมูลต้นทาง = ข้ามเอกสารนั้น ไม่ลบของเดิมทิ้ง
แถวนี้อัปได้หรือไม่ (prune)	<code>sgi_document_competitors .source_system</code>	ลบได้เฉพาะแถวที่ <code>source_system = 'ALLMAP'</code> และไม่มีอยู่ในชุดต้นทางปัจจุบันแล้ว	X แถว <code>source_system = 'USER'</code> (คนคีย์เอง) ห้ามลบทุกกรณี — รวมถึงตอนส่ง docNo มาเจาะจง
แถวซ้ำ	UNIQUE (doc_no, competitor_store_code)	upsert ด้วยคีย์นี้ — ค่าจากต้นทางชนะค่าที่ระบบเคยใส่ไว้	ชน unique = อัปเดตแถวเดิม ไม่ใช่ error

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว
ไม่ขึ้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
source_system	ALLMAP = ระบบนำเข้าให้ (ลบ/อัปเดตได้) · USER = คนคีย์เอง (ห้ามแตะ)	คอลัมน์ source_system ของ sgi_document_competitors

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
รันตามตารางเวลา	CRON	scheduler → runner (job 7)	อ่าน cron/พารามิเตอร์จาก backend config
รันนอกรอบ (manual/rerun)	CLI	CLI/ops runbook → runner (job 7)	guard ไม่ให้รันซ้อนด้วย distributed lock
แก้พารามิเตอร์/เปิด-ปิด job	CONFIG	แก้ backend config แล้ว deploy	ไม่มี endpoint และไม่มีหน้าจอบทความ — หน้า Flow Batch Job เป็น reference อย่างเดียว (2026-08-06)
ตรวจผลการรัน	LOG	application log (structured)	ไม่มีตาราง job_run_histories แล้ว · ผลการรับส่งไฟล์/ข้อความดูที่ sgi_interface_transactions

7. API Contract

เอกสารฉบับนี้ไม่มี endpoint ของตัวเอง — เป็นสัญญา/งานภายในที่เอกสารอื่นเรียกใช้ (ดูขอบเขตใน 5.90 Endpoint Implementation Contract) · รายการ endpoint ทั้ง 28 เส้นของ SGI อยู่ที่ LLDD-API.pdf และ API design

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช่งานสร้างหน้า Database, ไม่ใช่งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
sgi_fgi_impact_competitors	R	ข้อมูลคู่แข่งล่าสุดจาก Job 3
sgi_compensation_documents	R	หา doc_no จาก impact_process_id
sgi_document_competitors	W	บันทึกคู่แข่งเข้าเอกสารโดยตรง
sgi_interface_transactions	W	tracking ภายใน: direction=INTERNAL · status=COMPLETED (ไม่มี ACK ให้รอเพราะเขียน DB ตรง)

9. Skeleton Code (Batch Job 7)

9.1 ไฟล์ที่ต้องสร้าง (Job 7) — บน `srm-sps-spsap-sop-sgi-batch`

โครงไฟล์ของ Job 7 (document.service.syncCompetitors เดิม) วางได้ src/modules/sgi/ ตาม convention ที่ repo นี้ใช้อยู่จริง (ดูตัวอย่างที่ src/modules/performance/import-qssi.service.ts): service ถือ logic ทั้งหมด, inject DataSource/repository ผ่าน TypeORM, ยิง raw SQL ได้ตรง, และ ไม่มี controller เพราะ repo นี้ไม่เปิด HTTP

สิ่งที่ reuse ได้ทันที ไม่ต้องเขียนใหม่ — ต่างจากแผนเดิมที่ตั้งไว้บน store-backend ซึ่งต้องสร้าง runner ทั้งหมดเอง: src/main.ts (dispatcher + BATCH_START/BATCH_END + runId + log ลง integration_log อัตโนมัติ) · src/modules/rabbitMQ/rabbitmq.service.ts (publishMessage) · src/shared/services/s3.service.ts · StatementService.decodeThaiFileContent() (WINDOWS-874 auto-detect) · @gosoft-sbp/email-lib · @srm/glb-log

▲ สิ่งที่ repo นี้ยังไม่มี และเป็นงานตั้งต้นจริง: (1) ไม่มี @Cron เลย — ตารางเวลาต้องตั้งเป็น AWS Batch scheduled event (2) ไม่มี distributed lock (pg_try_advisory_lock) — การกันรันซ้อนพึ่ง AWS Batch queue ถ้างานไหนรับความเสี่ยงนี้ไม่ได้ต้องเพิ่มเอง (3) publishMessage เป็น fire-and-forget ไม่มี publisher confirm และไม่มี outbox — งานที่ต้องการ transactional outbox + publisher confirm (Job 6 · และ sgi_reflow ฝั่ง BE) ต้องสร้างกลไกเพิ่มเอง ไม่ใช่ reuse ได้เลย · ▲ ไม่มี ACK ระดับธุรกิจให้รอ (มติ 2026-09-08 ข้อ 2.13) (4) ยังไม่มี entity/ตาราง sgi_* แม้แต่ตัวเดียว

Path	หน้าที่
src/modules/sgi/job-7-sync-competitor-to-document.service.ts	คลาส SyncCompetitorToDocumentService — execute(input) เป็น entry point เดียว เรียงตาม flow ของ Job 7 ที่ละขั้น ครอบ transaction เอง และจบด้วย structured log สรุป metrics (แบบเดียวกับ ImportQssiService.execute(body) ที่มีอยู่)
src/modules/sgi/job-7-sync-competitor-to-document.service.spec.ts	unit test ของ service — repo นี้วาง spec ไว้ข้างไฟล์จริงเสมอ (jest + npm run test:ci มี coverage/SonarQube)
src/modules/sgi/dto/job-7-sync-competitor-to-document-input.dto.ts	DTO ของ INPUT (JSON) พร้อม class-validator ตามตารางในหัวข้อ 9.2 — parse ไม่ผ่านต้อง fail ก่อนแตะ DB
src/modules/sgi/sgi.module.ts	NestJS module ของกลุ่มงานประกันรายได้ — ผูก service ทุกตัวของ SGI เข้ากับ TypeOrmModule (ไฟล์รวมของทุก job ให้ merge ไม่ใช่เขียนทับ)
src/main.ts	เพิ่ม case 'sgi-sync-competitor-to-document': ในสวิตช์เดิม → await import('./modules/sgi/job-7-sync-competitor-to-document.service') แล้ว app.get(SyncCompetitorToDocumentService).execute(input) (ไฟล์กลางของทุก job — เป็นจุด merge conflict ที่ต้องระวัง)
src/entities/sgi-*.entity.ts	entity ของตาราง sgi_* ที่หัวข้อ Reference DB Mapping อ้างถึง — ยังไม่มีใน repo เลยสักตัว ต้องสร้างใหม่ทั้งหมด
src/config/config.ts	เพิ่ม export const sgiJob7Config ตามแบบของไฟล์นี้ (โปรเจกต์ไม่ใช่ registerAs) — ค่าคงที่ทางธุรกิจของ Job 7

การลงทะเบียนใน `src/main.ts` (job `sgi-sync-competitor-to-document`)

```
// src/main.ts — เพิ่มเคสในสวิตช์เดิม (เรียงต่อจาก job ของ SGI ตัวก่อนหน้า)
case 'sgi-sync-competitor-to-document': {
  const { SyncCompetitorToDocumentService } = await import('./modules/sgi/job-7-sync-competitor-to-document.service');
  const job7synccompetitortodocumentService = app.get(SyncCompetitorToDocumentService);
  await job7synccompetitortodocumentService.execute(input); // input = JSON ที่ parse จาก INPUT/argv[2] แล้ว
  break;
}
```

main.ts เรียก StatementService.logInterfest('sgi-sync-competitor-to-document', input) ให้อยู่แล้วก่อนเข้าสวิตช์ → ไม่ต้องเขียน log ลง integration_log เองซ้ำ · และ BATCH_END ที่ท้ายไฟล์จะสรุป batchSize + durationMs ให้อัตโนมัติ หน้าที่ของ service คือ throw เมื่อทำงานไม่สำเร็จเท่านั้น

9.2 Config Schema ของ Job 7 (backend config / env)

ตารางเวลาของ Job 7 คือ 0 18 7-31 * * (วันที่ 7-31 เวลา 18:00 — เลื่อนหลัง Job 8 (17:30) เพราะต้องรอ doc_no · ▲ ตั้งตั้ง dependency ที่ AWS Batch) — ▲ ตัวจริงตั้งที่ AWS Batch scheduled event ไม่ใช่ในโค้ด (repo นี้ไม่มี @Cron เลย) ค่า SGI_JOB7_CRON เก็บไว้เป็นเอกสารประกอบ/ตรวจสอบเท่านั้น · SGI_JOB7_ENABLED=false ให้ execute() จบทันทีแบบ SUCCESS พร้อม log เหตุผล (กันกรณี AWS Batch ยังยิงเข้ามา)

```
// src/config/config.ts — เพิ่มบล็อกนี้ต่อท้าย (repo ใช้ export const ไม่ใช่ registerAs)
// convention จริงของ sop-sgi-batch คือ `export const` ใน `src/config/config.ts` (ไม่ใช่ registerAs · ดูของเดิมที่
// `AppConfigModule` แบบ @Global แล้วอ่าน process.env ตรง ๆ) — โปรเจกต์นี้ **ไม่ได้ใช้ registerAs**
// แมแต่จุดเดียว จึงประกาศเป็นคลาสให้รีวิว/ทดสอบเหมือน config ตัวอื่น
import { Injectable } from '@nestjs/common';

// TODO: Job 7 ไม่มีตาราง job_configs และไม่มี Job Admin API แล้ว (ตัดสินใจ 2026-08-06)
// TODO: ค่าทุกตัวอ่านจาก env/config file ของ backend เท่านั้น — เปลี่ยนค่า = แก้ config แล้ว deploy
export interface Job7Config {
  /** เปิด/ปิด job รอบถัดไปโดยไม่ต้อง deploy โค้ด */
  enabled: boolean;
  /** ตารางเวลาของ job นี้ — บันทึกไว้เพื่ออ้างอิงเท่านั้น ตัวจริงตั้งที่ AWS Batch scheduled event */
  cron: string;
  /** Target table — upsert ด้วย doc_no / competitor_store_code · source_system = ALLMAP (ค่า canonical ค่าเดียวทั้งระบบ) */
  targetTable: string;
  /** เงื่อนไขเลือกข้อมูล — คง business rule เดิม */
  condition: string;
  /** ผู้รับอีเมลเมื่อ job ล้มเหลว — เก็บเป็น string คั่น comma ให้ตรง signature ของ
    `EmailLibService.sendMail({ mailTo })` ที่รับ string ไม่ใช่ string[] */
  mailTo: string;
}

@Injectable()
export class SgiJob7Config implements Job7Config {
  // TODO: ยืนยันค่า default ทุกตัวกับ Ops ก่อนขึ้น production (ไม่มีหน้าจอกำหนดค่าแล้ว)
  enabled = (process.env.SGI_JOB7_ENABLED ?? 'true') === 'true';
  cron = process.env.SGI_JOB7_CRON ?? '0 18 7-31 * *';
  targetTable = process.env.SGI_JOB7_TARGET_TABLE ?? 'sgi_document_competitors'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
}
```

```

condition = process.env.SGI_JOB7_CONDITION ?? 'งวดคู่แข่งล่าสุดต่อร้าน + forecast เริ่มต้น + ยังไม่ sync'; // TODO: ค่าคงที่ทางธุรกิจ —
เปลี่ยนต้องผ่านการอนุมัติ
mailTo = process.env.SGI_JOB7_MAIL_TO ?? ''; // TODO: ผู้รับอีเมลแจ้ง error คั่นด้วย comma (เดิม: ส่ง error ผ่าน email-lib กลาง (sendEmail) เมื่อ
sync ลมเหลว)
}

// TODO: เพิ่ม SgiJob7Config ใน providers/exports ของ AppConfigModule (@Global) เหมือน AppConfig

```

9.3 Job Class — `run(ctx)` ของ Job 7 ที่ละชั้นตามผัง

9.3.1 สัญญาของชั้นกลาง (`runner.ts`) + โครง service ของ Job 7

service อ้าง JobRunContext / JobRunResult / JobState / JobFailedError — ทั้งหมดนิยาม ครั้งเดียวใน src/modules/sgi/sgi-job.types.ts (ไฟล์ร่วมของทุก job ให้ merge ไม่ใช่เขียนทับ) และ service ต้องมี method ครบตามตารางชั้นตอนด้านล่าง มิฉะนั้น execute(input) จะเรียก method ที่ไม่มีอยู่

```

// src/modules/sgi/sgi-job.types.ts — สัญญากลางของทุก job ของ SGI (ประกาศครั้งเดียว ใช้ร่วมทั้ง 10 ฉบับ)

export interface JobRunContext {
  jobNo: string;
  period: string; // YYYYMM ของงวดที่รัน
  triggeredBy: string; // 'CRON' | userId ที่สั่งรันนอกรอบ
  params?: Record<string, string>;
}

export interface JobRunResult {
  event: 'job.finish';
  jobNo: string;
  jobName: string;
  status: 'SUCCESS' | 'SKIPPED' | 'SKIPPED_LOCKED' | 'FAILED';
  period: string;
  output: string;
  read: number; written: number; skipped: number; rejected: number;
  durationMs: number;
}

/** counter + ค่าที่ทุกชั้นของ job ใช้ร่วมกัน (service เป็นผู้สร้างผ่าน createState) */
export interface JobState {
  period: string;
  read: number; written: number; skipped: number; rejected: number;
  // TODO: เพิ่ม field เฉพาะของ job นี้ (เช่น rows ที่อ่านมา, path ไฟล์ที่เขียน)
  [key: string]: unknown;
}

/** error ที่ทำให้ job จบเป็น FAILED และส่งอีเมลแจ้งผู้ดูแล */
export class JobFailedError extends Error {
  constructor(public readonly code: string, message: string) { super(message); }
}

/** ไข่ออกจาก transaction เมื่อสาขา NO บอกให้ข้ามงวด/เรคคอร์ด — runner สรุปรเป็น SKIPPED ไม่ใช่ FAILED */
export class JobSkippedError extends Error {}

// SyncCompetitorToDocumentService — method ที่ job class เรียก (1 method ต่อ 1 ชั้นในตารางด้านบน)
import { Inject, Injectable } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import type { JobRunContext, JobState } from '../runner';
export type { JobState };

@Injectable()
export class SyncCompetitorToDocumentService {
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  createState(ctx: JobRunContext): JobState {
    return { period: ctx.period, read: 0, written: 0, skipped: 0, rejected: 0 };
  }

  // อ่านคู่แข่งงวดล่าสุดต่อร้านจาก sgi_fgi_impact_competitors
  async step02Read(state: JobState, manager?: EntityManager): Promise<void> {
    throw new Error('step02Read: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกากำหนดที่หัวข้อเงื่อนไขตัดสินใจ');
  }

  // มี sgi_compensation_documents ของ impact_process_id แล้ว?
  // เงื่อนไขจริง: ดูตารางในหัวข้อ "เงื่อนไขตัดสินใจ (Decision Rules)" ของเอกสารฉบับนี้
  async check03Document(state: JobState): Promise<boolean> {
    throw new Error('check03Document: ยังไม่ได้ implement — ห้าม deploy ทั้งที่ยังไม่เขียนเงื่อนไขจริง');
  }

  // upsert sgi_document_competitors
  async step04Upsert(state: JobState, manager?: EntityManager): Promise<void> {
    throw new Error('step04Upsert: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกากำหนดที่หัวข้อเงื่อนไขตัดสินใจ');
  }
}

```

```
// insert sgi_interface_transactions: data_name = IMPACT_COMPETITOR · direction = INTERNAL · status = COMPLETED
async step05WriteFile(state: JobState, manager?: EntityManager): Promise<void> {
  throw new Error('step05WriteFile: ยังไม่ได้ implement — ดู SQL ที่หัวข้อ Repository / SQL และกติกาส่งที่หัวข้อเงื่อนไขตัดสิน');
}
}
```

9.3.2 `run(ctx)` ของ Job 7

ทุกขั้นใน run() ตรงกับ flowchart ของ Job 7 หนึ่งต่อหนึ่ง (decision และ error path รวมอยู่ด้วย) — method ที่ต้อง implement ใน service ตามตารางนี้

ลำดับ	ชนิด	ขั้นตอนจากผัง	Method ที่ต้อง implement	เส้นทาง NO / error
1	start	เริ่ม	createState()	-
2	process	อ่านคู่แข่งงวดล่าสุดต่อร้านจาก sgi_fgi_impact_competitors	step02Read()	throw JobFailedError เมื่อทำไม่สำเร็จ
3	decision	มี sgi_compensation_documents ของ impact_process_id แล้ว?	check03Document()	[บันทึกผลแล้วไป record ถัดไป] คงสถานะรอ sync / log pending
4	process	upsert sgi_document_competitors	step04Upsert()	throw JobFailedError เมื่อทำไม่สำเร็จ
5	process	insert sgi_interface_transactions: data_name = IMPACT_COMPETITOR · direction = INTERNAL · status = COMPLETED	step05WriteFile()	throw JobFailedError เมื่อทำไม่สำเร็จ
6	end	จบ	summarize()	-

```
// src/modules/sgi/job-7-sync-competitor-to-document.service.ts — execute(input) เป็น entry point เดียว
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import { SyncCompetitorToDocumentService, type JobState } from './job-7-sync-competitor-to-document.service';
// 4 symbol นี้นิยามใน src/modules/sgi/sgi-job.types.ts (ไฟล์รวมของทุก job — ดูหัวข้อก่อนหน้า)
import { JobFailedError, JobSkippedError, JobRunContext, JobRunResult } from '../../runner';

@Injectable()
export class SyncCompetitorToDocumentJob {
  static readonly jobNo = '7';
  private readonly logger = new Logger(SyncCompetitorToDocumentJob.name);

  constructor(
    // TODO: repo นี้ตั้ง replication ใน typeorm.config.ts อยู่แล้ว — SELECT ไป slave, write ไป master อัตโนมัติ
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly service: SyncCompetitorToDocumentService,
  ) {}

  async run(ctx: JobRunContext): Promise<JobRunResult> {
    const startedAt = Date.now();
    // TODO: state ถือ candidates ที่อ่านมา + counter (read/written/skipped/rejected/merged)
    // และค่าจาก job7Config — ทุก counter ต้องถูกอัปเดตจาก record จริง ไม่ใช่ค่าคงที่
    const state = this.service.createState(ctx);
    try {
      // ขั้นที่ 2: อ่านคู่แข่งงวดล่าสุดต่อร้านจาก sgi_fgi_impact_competitors · TODO: dense rank ตามงวดต้นทางของคู่แข่ง
      await this.service.step02Read(state);
      // TODO: candidate มาจากขั้นอ่านข้อมูลด้านบน — ลูปนี้จำเป็นเพราะมี branch ระดับ record
      // (ขั้นที่ตัดสินรายแถวจะ 'continue'/'return' ออกจากรอบของ record นั้น)
      // เชื่อมบรรทัดในลูปให้เรียบรอยตอนคัดลอกเข้าไปรอกดจริง
      for (const record of state.candidates) {
        // ขั้นที่ 3 (decision): มี sgi_compensation_documents ของ impact_process_id แล้ว?
        const ok03 = await this.service.check03Document(state);
        if (!ok03) { // NO → คงสถานะรอ sync / log pending
          await this.service.mark03(state);
          state.marked += 1;
          continue; // ไม่ record ถัดไป — ไม่ใช่ error ของทั้ง job
        }
      }
      // === transaction boundary === TODO: DB transaction ครอบการ upsert sgi_document_competitors + tracking
      await this.dataSource.transaction(async (manager: EntityManager) => {
        // ขั้นที่ 4: upsert sgi_document_competitors · TODO: source_system = ALLMAP · ผูก doc_no และ competitor_store_code · คัดลอก brand_code มาด้วย
        await this.service.step04Upsert(state, manager);
        // ขั้นที่ 5: insert sgi_interface_transactions: data_name = IMPACT_COMPETITOR · direction = INTERNAL · status = COMPLETED ·
        // TODO: ไม่สร้างไฟล์ BPM060030 แล้ว — เขียน DB ตรงจึงไม่มี ACK ให้รอ
      });
    } catch (error) {
      // ...
    }
  }
}
```

```

    await this.service.step05WriteFile(state, manager);
  });
}
return this.summarize(state, 'SUCCESS', startedAt);
} catch (error) {
  // TODO: error path ของ Job 7 — ห้าม re-implement การเขียนไฟล์ BPM06003O หรือ SFTP ไป BPM; legacy file เป็น reference เท่านั้น
  this.logger.error(JSON.stringify({ event: 'job.failed', jobNo: '7', period: ctx.period,
    triggeredBy: ctx.triggeredBy, durationMs: Date.now() - startedAt, error: (error as Error).message }));
  // TODO: แจ้งผู้ดูแลผ่าน JobFailureNotifier (หัวข้อ 9.6.1) — runner เป็นผู้เรียกให้
  throw error;
}
}
}

private summarize(state: JobState, status: JobRunResult['status'], startedAt = Date.now()): JobRunResult {
  // TODO: structured log บรรทัดเดียวจบ — ไม่มีตาราง job_run_histories แล้ว (2026-08-06)
  const summary = {
    event: 'job.finish', jobNo: '7', jobName: 'SyncCompetitorToDocument', status,
    period: state.period, output: 'sgi_document_competitors (DB)',
    read: state.read, written: state.written, skipped: state.skipped,
    rejected: state.rejected, durationMs: Date.now() - startedAt,
  };
  this.logger.log(JSON.stringify(summary));
  return summary as JobRunResult;
}
}
}

```

9.4 การกั้นรันชอนของ Job 7 (PostgreSQL advisory lock — **ของใหม่**)

Job 7 มีข้อควรระวังจาก legacy: ห้าม re-implement การเขียนไฟล์ BPM06003O หรือ SFTP ไป BPM; legacy file เป็น reference เท่านั้น — runner ล็อกด้วย pg_try_advisory_lock ก่อนเริ่มรันแรกเสมอ และรอบที่ล็อกไม่ได้ให้จบด้วยสถานะ SKIPPED_LOCKED (ไม่ใช่ FAILED)

```

// src/modules/sgi/sgi-job.lock.ts (ของใหม่ — repo นี้ยังไม่มีการกั้นรันชอนใด ๆ)
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource } from 'typeorm';

// TODO: ห้ามใช้แถวสถานะ RUNNING ในตารางเป็นตัวกัน (ไม่มีตาราง job_run_histories แล้ว)
// ใช้ PostgreSQL advisory lock ระดับ session แทน — ปลดล็อกอัตโนมัติเมื่อ connection หลุด
export const SGI_JOB_LOCK_CLASS_ID = 861000; // namespace ของระบบ SGI
export const JOB_LOCK_KEYS: Record<string, number> = { '7': 70 /* TODO: เพิ่มให้ครบทุก job */ };

@Injectable()
export class BatchRunner {
  private readonly logger = new Logger(BatchRunner.name);
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  // period = งวดที่รอบนี้ทำงาน ('YYYY-MM') — เป็นส่วนหนึ่งของคีย์ล็อก ไม่ใช่แค่หมายเลข job
  // (เจอจริง 2026-09-09: ล็อกด้วย jobNo อย่างเดียว = คนละงวดก็รันพร้อมกันไม่ได้)
  // ทั้งที่เอกสารระบุว่าคนละงวดต้องรันขนานกันได้ · ส่ง period = null ถ้าต้องการล็อกทั้ง job)
  async runExclusive<T>(jobNo: string, period: string | null, fn: () => Promise<T>): Promise<T | { status: 'SKIPPED_LOCKED' }> {
    // TODO: ต้องใช้ QueryRunner (connection เดียวบน master) — dataSource.query() ของโปรเจกต์นี้
    // route SQL ที่ขึ้นต้นด้วย SELECT ไป slave pool ทำให้ lock ไปตกที่ replica คนละ connection
    const runner = this.dataSource.createQueryRunner('master');
    await runner.connect();
    // pg_try_advisory_lock(int4, int4) — objectId ต้องอยู่ในช่วง int4
    // ล็อกทั้ง job : objectId = JOB_LOCK_KEYS[jobNo]
    // ล็อกรายงวด : ผสมงวดเขาไปด้วย hashtext() แล้วบีบให้อยู่ในช่วงที่ปลอดภัย
    const baseId = JOB_LOCK_KEYS[jobNo];
    const objectId = period === null ? baseId
      : (await runner.query('SELECT (hashtext($1) & 2147483647) % 1000000 + $2 * 1000000 AS id',
        [period, baseId]))[0].id;
    try {
      const [{ locked }] = await runner.query(
        'SELECT pg_try_advisory_lock($1, $2) AS locked',
        [SGI_JOB_LOCK_CLASS_ID, objectId],
      );
      if (!locked) {
        // TODO: รอบนี้ข้ามไปเลย ๆ ไม่ถือเป็น error และไม่ต้องส่งอีเมล
        this.logger.warn(JSON.stringify({ event: 'job.skipped.locked', jobNo, period }));
        return { status: 'SKIPPED_LOCKED' };
      }
    }
    return await fn();
  } finally {
    // TODO: ปลด lock ทุกกรณี แล้วคืน connection เข้า pool
    await runner.query('SELECT pg_advisory_unlock($1, $2)', [SGI_JOB_LOCK_CLASS_ID, objectId]);
    await runner.release();
  }
}
}

```

9.5 Repository / SQL หลักของ Job 7

repository ของ Job 7 ประกาศเป็น factory provider ({provide: 'SYNC_COMPETITOR_TO_DOCUMENT_REPOSITORY', useFactory: (ds) => ds.getRepository(Entity), inject: [DataSource]}) แล้วยิง raw SQL ตามแบบ module ธุรกิจอื่นของ store-backend (schema sgs_store มาจาก search_path)

ตาราง	R/W	การใช้งานตามผัง	หมายเหตุ target design
sgi_fgi_impact_competitors	R	ข้อมูลคู่แข่งล่าสุดจาก Job 3	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_compensation_documents	R	หา doc_no จาก impact_process_id	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_document_competitors	W	บันทึกคู่แข่งเข้าเอกสารโดยตรง	เขียน SQL ตรงผ่าน DATA_SOURCE
sgi_interface_transactions	W	tracking ภายใน: direction=INTERNAL · status=COMPLETED (ไม่มี ACK ให้รอเพราะเขียน DB ตรง)	เขียน SQL ตรงผ่าน DATA_SOURCE

```
-- Job 7 SyncCompetitorToDocument — query หลักที่ต้อง implement
-- TODO: ทุก statement รันผ่าน DATA_SOURCE (SELECT ไป slave, write ไป master) และ
-- write ทั้งหมดต้องอยู่ใน transaction เดียวกันที่ระบุใน 9.3

-- [R] sgi_fgi_impact_competitors : ข้อมูลคู่แข่งล่าสุดจาก Job 3
-- คอลัมน์มาจาก DDL จริงของตารางนี้ (ห้าม SELECT *) · ตรวจสอบว่ามี index รองรับ WHERE ก่อนขึ้น prod
SELECT id, branch_th, brand_code, closed_date, competitor_key, competitor_store_code, impact_process_id, name_en, name_th,
opened_date, period_key, subzone_code -- คัดคอลัมน์ที่ job นี้ไม่ได้ให้ออก (ทั้งตารางมี 14 คอลัมน์)
FROM sgi_fgi_impact_competitors
WHERE period_key = $1 -- คอลัมน์จริงของตารางนี้ตาม DDL
ORDER BY id -- PK ทำให้ลำดับคงที่ระหว่างแบ่งหน้า
LIMIT $2 OFFSET $3; -- อ่านเป็น chunk กัน memory บวม

-- [R] sgi_compensation_documents : หา doc_no จาก impact_process_id
-- คอลัมน์มาจาก DDL จริงของตารางนี้ (ห้าม SELECT *) · ตรวจสอบว่ามี index รองรับ WHERE ก่อนขึ้น prod
SELECT id, account_month, account_year, allmap_url, approver_snapshot, created_at, created_by, current_section_code, doc_no,
impact_compensation_id, impact_month, impact_process_id -- คัดคอลัมน์ที่ job นี้ไม่ได้ให้ออก (ทั้งตารางมี 26 คอลัมน์)
FROM sgi_compensation_documents
WHERE impact_month = $1 -- คอลัมน์จริงของตารางนี้ตาม DDL
ORDER BY id -- PK ทำให้ลำดับคงที่ระหว่างแบ่งหน้า
LIMIT $2 OFFSET $3; -- อ่านเป็น chunk กัน memory บวม

-- [W] sgi_document_competitors : บันทึกคู่แข่งเข้าเอกสารโดยตรง
-- คอลัมน์มาจาก DDL จริง — คัดคอลัมน์ที่ job นี้ไม่ได้เขียนออก แล้วเลื่อนเลข $n ให้ตรง
INSERT INTO sgi_document_competitors
(doc_no, source_system, branch_th, brand_code, closed_date, competitor_key, competitor_store_code, detail, impact_date, name_en,
name_th, opened_date, remark, source_row_id)
VALUES ($1 /* doc_no */, $2 /* source_system */, $3 /* branch_th */, $4 /* brand_code */, $5 /* closed_date */, $6 /* competitor_key */, $7 /*
competitor_store_code */, $8 /* detail */, $9 /* impact_date */, $10 /* name_en */, $11 /* name_th */, $12 /* opened_date */, $13 /* remark */, $14
/* source_row_id */)
ON CONFLICT (doc_no, competitor_store_code) -- unique key จริงตาม DDL ของ sgi_document_competitors (ห้ามเอา)
DO UPDATE SET source_system = EXCLUDED.source_system, branch_th = EXCLUDED.branch_th, brand_code = EXCLUDED.brand_code,
closed_date = EXCLUDED.closed_date, competitor_key = EXCLUDED.competitor_key, detail = EXCLUDED.detail, impact_date =
EXCLUDED.impact_date, name_en = EXCLUDED.name_en, name_th = EXCLUDED.name_th, opened_date = EXCLUDED.opened_date,
remark = EXCLUDED.remark, source_row_id = EXCLUDED.source_row_id, subzone_code = EXCLUDED.subzone_code, zone_code =
EXCLUDED.zone_code,
updated_at = NOW();

-- [W] sgi_interface_transactions : tracking ภายใน: direction=INTERNAL · status=COMPLETED (ไม่มี ACK ให้รอเพราะเขียน DB ตรง)
-- บันทึกผลการรับส่งระดับ record ของ interface (แทน job_run_histories ที่ยกเลิกไปแล้ว)
INSERT INTO sgi_interface_transactions
(run_id, data_name, direction, status, business_key, period_key,
file_name, file_checksum, created_at)
VALUES ($1 /* run_id = correlation id ของรอบรัน Job 7 จาก application log */,
$2 /* TODO: data_name ของ Job 7 */, $3 /* IN|OUT|INTERNAL */, 'READY',
$4 /* business key ของแถว */, $5 /* YYYYMM */, $6, $7, NOW())
ON CONFLICT (data_name, direction, business_key, period_key) DO NOTHING;
```

9.6 การแจ้งเตือนและการรันซ้ำของ Job 4

9.6.1 อีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว

ใช้ EmailLibService จาก @gosoft-sbp/email-lib ตัวเดียวกับที่ระบบเดิมใช้ (inform-evaluate / external-audit / statement PTT) — ไม่สร้างกลไกส่งเมลใหม่

```
// src/modules/sgi/sgi-job-failure.notifier.ts (ใช้ @gosoft-sbp/email-lib ที่ repo มียอยู่แล้ว)
import { Injectable, Logger } from '@nestjs/common';
```

```
// ชื่อ method ของ lib ที่ repo นี้เรียกจริงคือ 'sendMail' (ไม่ใช่ sendEmail) และ
// 'mailTo' / 'mailCc' เป็น **string** คั่นด้วย comma — ดู evaluation-process.service.ts,
// external-audit.service.ts, statement.service.ts, inform-evaluate.service.ts, performance.service.ts
import { EmailLibService } from '@gosoft-sbp/email-lib';
import type { JobRunContext } from './runner';

@Injectable()
export class JobFailureNotifier {
  private readonly logger = new Logger(JobFailureNotifier.name);
  // TODO: ใช้ lib อินลิของระบบเดิม — template อยู่ในตาราง email_template และ log ลง email_sent อัตโนมัติ
  // (ตั้งชื่อ property ว่า mailService ตาม call site เดิมของ sop-sgi-batch)
  constructor(private readonly mailService: EmailLibService) {}

  async notifyFailure(jobNo: string, ctx: JobRunContext, error: Error): Promise<void> {
    // TODO: ผู้รับของ Job 7 เดิมคือ ส่ง error ผ่าน email-lib กลาง (sendEmail) เมื่อ sync ล้มเหลว — ย้ายมาเป็น env SGI_JOB7_MAIL_TO
    const recipients = (process.env.SGI_JOB7_MAIL_TO ?? '').split(',').map(s => s.trim()).filter(Boolean);
    if (!recipients.length) {
      this.logger.warn(JSON.stringify({ event: 'job.mail.skipped', jobNo, reason: 'NO_RECIPIENT' }));
      return;
    }
    try {
      await this.mailService.sendMail({
        // TODO: emailId = id ของ template EM-07 (แจ้ง error batch) ในตาราง email_template
        emailId: Number(process.env.SGI_JOB_FAIL_EMAIL_TEMPLATE_ID),
        mailTo: recipients.join(','), // signature รับ string ไม่ใช่ string[]
        mailCc: "",
        param: {
          jobNo, jobName: 'SyncCompetitorToDocument',
          jobTitle: 'บันทึกข้อมูลคู่แข่งเอกสาร',
          period: ctx.period, triggeredBy: ctx.triggeredBy,
          output: 'sgi_document_competitors (DB)',
          errorMessage: error.message,
          rerunNote: 'idempotent ด้วย doc_no + competitor_store_code + source_system (ALLMAP)',
        },
      });
    } catch (mailError) {
      // TODO: ส่งเมลไม่สำเร็จห้ามกลับ error เดิมของ job — log แล้วปล่อยผ่าน
      this.logger.error(JSON.stringify({ event: 'job.mail.failed', jobNo, error: (mailError as Error).message }));
    }
  }
}
```

9.6.2 Checklist การ rerun

- กติกา rerun ของ Job 7: idempotent ด้วย doc_no + competitor_store_code + source_system (ALLMAP)
- ขอบเขต transaction ที่ต้องรักษาเมื่อรันซ้ำ: DB transaction ครอบคลุมการ upsert sgi_document_competitors + tracking
- ความเสี่ยงที่ต้องตรวจก่อน/หลังรันซ้ำ: ห้าม re-implement การเขียนไฟล์ BPM06003O หรือ SFTP ไป BPM; legacy file เป็น reference เท่านั้น
- ตรวจสอบว่ารอบก่อนหน้าไม่ได้ค้าง lock อยู่ (SELECT * FROM pg_locks WHERE locktype = 'advisory') ก่อนสั่งรันนอกรอบ
- สั่งรันนอกรอบผ่าน CLI/runbook เท่านั้น (ไม่มีหน้าจอสั่งและไม่มี Job Admin API) — local:
JOB_NAME=sgi-sync-competitor-to-document INPUT='{ "year":2026,"month":6 }' npm run start ·
AWS Batch: node dist/main.js '{ "year":2026,"month":6 }' sgi-sync-competitor-to-document
(quote เดี่ยวครอบ JSON เสมอ) · ตรวจสอบด้วย echo \$? ต้องเป็น 0 เมื่อสำเร็จ
- หลังรันซ้ำ ตรวจสอบ output sgi_document_competitors (DB) และ log บรรทัด job.finish ว่า
read/written/skipped/rejected ตรงกับที่คาด
- ถ้ารอบก่อนล้มเหลวกลางทาง ตรวจสอบ sgi_interface_transactions ของงวดนั้นว่ามีแถวค้างสถานะ READY/PENDING หรือไม่ ก่อนสั่งรันใหม่

10. Processing Flow

Step	Description
1	เริ่ม
2	อ่านคู่แข่งงวดล่าสุดต่อร้านจาก sgi_fgi_impact_competitors (dense rank ตามงวดต้นทางของคุณแข่ง)

Step	Description
3	มี sgi_compensation_documents ของ impact_process_id แล้ว? No: คงสถานะรอ sync / log pending
4	upsert sgi_document_competitors (source_system = ALLMAP · ผูก doc_no และ competitor_store_code · คัดลอก brand_code มาด้วย)
5	insert sgi_interface_transactions: data_name = IMPACT_COMPETITOR · direction = INTERNAL · status = COMPLETED (ไม่สร้างไฟล์ BPM06003O แล้ว — เขียน DB ตรงจึงไม่มี ACK ให้รอ)
6	จบ

11. Acceptance Criteria

- พารามิเตอร์รับผ่าน INPUT (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ @Cron ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนทั้ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (BATCH_START/BATCH_END + runId — src/main.ts ทำให้แล้ว) และบันทึก integration_log อัปเดตในมิติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

12. Developer Test Checklist

No	Test
1	รันตามตารางเวลาแล้วผลถูกต้องบน fixture
2	รันนอกรอบผ่าน CLI ได้ผลเดียวกับ cron
3	สั่งรันซ้อนขณะกำลังรัน → runner ปฏิเสธ (lock ทำงาน)
4	แก้ config แล้ว deploy → รอบถัดไปใช้ค่าใหม่
5	job throw error → EM-07 ออก และ log มีบรรทัด error
6	ตรวจผลกระทบตารางตาม R/W mapping reference

13. Unit Test Scope

3 ชั่วโมง (30% ของ implementation 9 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
เงื่อนไขเลือกข้อมูล	rule	ใช้กฎกับข้อมูลตัวอย่างแล้วได้ผลตามที่ระบุ — งดคู่แข่งล่าสุดต่อร้าน + forecast เริ่มต้น + ยังไม่ sync
business rule	logic	พารามิเตอร์รับผ่าน INPUT (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ @Cron ในโค้ด
business rule	logic	การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนทั้ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
business rule	logic	ทุกรอบต้องเขียน application log แบบ structured (BATCH_START/BATCH_END + runId — src/main.ts ทำให้แล้ว) และบันทึก integration_log อัตโนมัติ · error ต้องส่ง EM-07
business rule	logic	DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช้งานสร้างหน้า Database
business rule	logic	รองรับ rerun rule และ risk note ตาม runbook
sgi_document_competitors, sgi_interface_transactions	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
runner	idempotency	รันซ้ำด้วย fixture เดิมต้องไม่เกิดแถวซ้ำ (ON CONFLICT / business unique key ทำงาน)
runner	lock	เรียกซ้อนขณะกำลังรัน ต้องถูกปฏิเสธด้วย advisory lock

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportCompetitor.java — lines 9-20

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportCompetitor.java
Original lines	9-20
Responsibility	Legacy main entrypoint for competitor export.

```
public class ExportCompetitor {  
  
    private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");  
    private static ExportController exportController = (ExportController) context.getBean("exportController");  
  
    public static void main(String[] args) {  
        LogUtils.info(ExportCompetitor.class,"Start => export IMPACT_COMPETITOR to BPM");  
        exportController.exportCompetitorToBPM();  
        LogUtils.info(ExportCompetitor.class,"End => export IMPACT_COMPETITOR to BPM");  
    }  
  
}
```

J2. ExportController.java — lines 659-670

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	659-670
Responsibility	Query competitor data, generate file content, upload, backup, notification.

```
public void exportCompetitorToBPM() {  
    String fileName = "";  
    Calendar calStart = Calendar.getInstance(Locale.US);  
    EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();  
    informBean.setSystemCode(FgiConstant.SYSTEM_CODE);  
    informBean.setJobName(FgiConstant.JOB_NAME_EXPORT_COMPETITOR);  
    informBean.setStartDateTime(calStart.getTime());  
    informBean.setImportFileName("");  
    try {  
        configFtp = (FtpBean) context.getBean("fgiExportCompetitor");  
        setValueConfig();  
        List<Map> competitorList = exportService.queryCompetitorToBPM();
```

J2. ExportController.java — lines 749-760

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	749-760
Responsibility	Query competitor data, generate file content, upload, backup, notification.

```
+ "file_name=" + fileName+ "|total line=" +countLine);

exportService.insertConfirmReceiveData(confirmReceiveDataList);

// upload to server EAI
FtpUtils.uploadSFTPFile(ftpEaiServerIpBpm, ftpEaiUserIdBpm, ftpEaiPasswordBpm, 22, null, 0, ftpSourcePath, ftpDestinationPath, ftpFileNameArray);////////
FileUtils.moveBackupFile(fileExport, fileBackup);
informBean.setStatus(FgiConstant.JOB_STATUS_SUCCESS);
    return new Boolean("true");
} catch (Exception ex){
LogUtils.error(getClass(),ex);
LogUtils.info(getClass(),"===== file isDelete = " + FileUtils.deleteFile(fileExport));
```

J3. ExportJdbc.java — lines 1596-1607

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1596-1607
Responsibility	Query latest competitor rows eligible for export.

```
public List<Map> queryCompetitorToBPM(){
StringBulder sql = new StringBulder();
sql.append(" select * ");
sql.append(" from ( ");
sql.append(" select fic.impact_competitor_id, fic.storecode_i, fic.compet_id, fic.name_th, fic.name, ");
sql.append(" fic.branch_th, fic.zone_code, fic.subzone_code, fic.open_date, fic.close_date, ");
sql.append(" op.last_compensate_month as month, op.last_compensate_year as year, ");
sql.append(" dense_rank() over(partition by fic.storecode_i order by fic.year desc, fic.month desc) as rank_period, ");
sql.append(" 'system' as create_by, fic.create_date, op.impact_process_id ");
sql.append(" from fgi_impact_competitor fic ");
sql.append(" inner join fgi_impact_store_on_process op on op.flag_action in ('"+FgiConstant.FLAG_ACTION_ACTION+"', '"+FgiConstant.FLAG_VERIFY_WAIT+"') ");
sql.append(" and op.datasource = 'ALM' ");
```

J3. ExportJdbc.java — lines 1617-1628

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1617-1628
Responsibility	Query latest competitor rows eligible for export.

```
sql.append(" and not exists ( ");
sql.append(" select rd.transaction_pk ");
sql.append(" from fgi_confirm_receive_data rd ");
sql.append(" inner join fgi_impact_competitor fic1 on rd.data_name = '"+FgiConstant.DATA_IMPACT_COMPETITOR+"' ");
sql.append(" and fic1.impact_competitor_id = rd.transaction_pk ");
sql.append(" where fic1.storecode_i = a.storecode_i ");
sql.append(" and rd.month = a.month ");
sql.append(" and rd.year = a.year ");
List<Map> impactCompetitorList = jdbcTemplate.queryForList(sql.toString());
LogUtils.info(getClass(),"query FGI_IMPACT_COMPETITOR for export: "+impactCompetitorList.size());
return impactCompetitorList;
}
```